

Technical Documentation — Solari Solar AI Chatbot

Selected Technologies, Architecture, Implementation, APIs, and Deployment

Prepared by: Taiba
August 2026

Technical Documentation Task

Scope: documentation of the selected technologies and technical implementation of the Solari project.

1. Document Control

Item	Value
Project	Solari — Solar AI Chatbot
Document	Technical Documentation
Prepared by	Taiba
Purpose	Document the technologies, architecture, implementation, APIs, data flow, deployment, and technical considerations.
Source package	Solar-AI-Chatbot-main.zip
Document date	August 2026

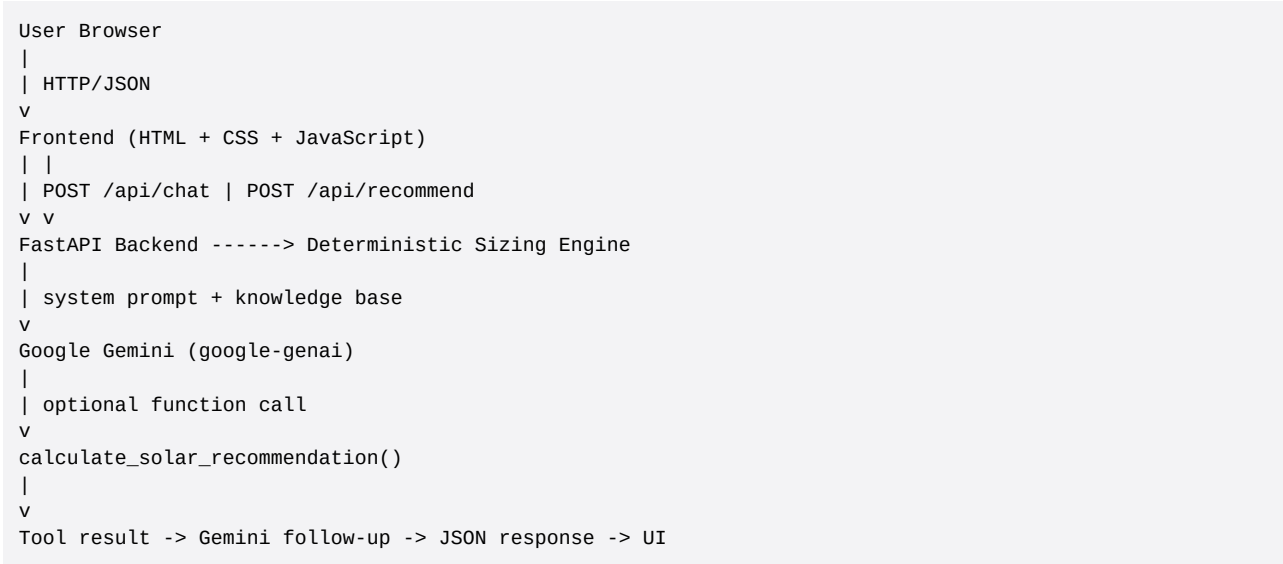
2. Project Overview

Solari is a web-based solar consultation assistant designed for residential, commercial, and industrial solar PV guidance in Pakistan. The project combines a browser-based interface, a Python FastAPI backend, a deterministic solar-sizing engine, a grounded knowledge base, and Google Gemini AI with native function calling. The system is designed so that conversational answers are grounded in project knowledge while numerical sizing is delegated to a deterministic calculation function.

The implementation is split into two primary application layers: a frontend in the frontend/ directory and a Python backend in the backend/ directory. The backend owns the AI integration, knowledge-base loading, API contracts, and deterministic sizing logic.

3. System Architecture

The implemented high-level architecture is:



The architecture deliberately separates conversational AI from deterministic engineering calculations. Gemini handles natural-language understanding and response generation, while the Python recommendation function produces the numeric sizing result used by the assistant.

4. Technology Stack

Layer	Technology	Role
Frontend	HTML5, CSS3, Vanilla JavaScript	Responsive single-page interface, chat UI, calculator modal, navigation

Layer	Technology	Role
UI Styling	Custom CSS	Design tokens, layouts, animations, responsive behavior, dark visual system
Client Markdown	Marked.js	Converts AI Markdown responses into HTML
Client Sanitization	DOMPurify	Sanitizes generated HTML before insertion into the page
Backend API	Python + FastAPI	REST endpoints, validation, CORS, lifecycle initialization
ASGI Server	Uvicorn	Runs the FastAPI application
Data Validation	Pydantic v2	Validates API request bodies and field constraints
Configuration	python-dotenv	Loads Gemini configuration from backend .env
AI SDK	google-genai	Gemini API client and native function/tool calling
AI Model	gemini-3.1-flash-lite	Configured conversational model
Knowledge Base	Markdown + PDF documents	Solar technical, operational, hardware, and regulatory grounding
Sizing Engine	Python deterministic formulas	System kW, panel count, roof area, and battery estimate

5. Frontend Technical Implementation

- The frontend is implemented without React, Vue, Angular, or another JavaScript framework; it uses standard HTML, CSS, and JavaScript.
- The main interface contains Chat, Knowledge Base, and Test Cases navigation areas.
- The chat composer accepts up to 500 characters and sends JSON to the FastAPI chat endpoint.
- Suggestion chips can trigger predefined requests such as system sizing, panel comparison, installation, and human support.
- A typing indicator and network-error banner provide basic request-state feedback.
- AI responses are parsed as Markdown using Marked.js and sanitized with DOMPurify before being inserted into the DOM.
- The system calculator collects monthly electricity usage and backup hours, then calls the deterministic recommendation endpoint.
- The Knowledge Base area exposes three project PDF documents, while the Test Cases area exposes the Excel test workbook.
- The current JavaScript uses local API URLs at 127.0.0.1:8000; these URLs must be changed or environment-configured for a deployed environment.

5.1 Frontend Structure

Path	Purpose
frontend/index.html	Page structure, navigation, chat interface, knowledge-base cards, test-case card, calculator modal.
frontend/css/style.css	Visual design system, layout, typography, responsive behavior, animations, component styling.
frontend/js/script.js	API integration, chat state, calculator requests, navigation tabs, Markdown rendering, error handling.
frontend/assets/	Logo, favicon, web manifest, and application icon assets.

6. Backend Technical Implementation

- FastAPI initializes the application and loads the knowledge base and system prompt during application lifespan startup.
- CORS middleware is enabled to allow browser access to the API.
- The backend defines /health and / routes for service status.
- POST /api/recommend performs deterministic sizing without invoking the AI model.
- POST /api/chat handles conversational requests and can invoke the solar-sizing function through Gemini native tool calling.
- Pydantic models validate monthly_units, backup_hours, message, and system_type.
- The backend returns JSON responses with success status, message text, and, when applicable, tool_result data.
- The application is intended to run on Uvicorn at port 8000.

6.1 Backend Modules

Module	Responsibility
main.py	FastAPI application, lifespan initialization, middleware, request models, routes, Gemini orchestration.
utils.py	Knowledge-base loading, system-prompt loading, deterministic sizing engine, Gemini tool schema.
config.py	Environment loading and application/Gemini configuration.
data/	Markdown knowledge-base source files.
prompts/system_prompt.txt	Domain behavior, grounding rules, safety rules, sizing protocol, response style.
knowledge_base/	Project PDF reference documents exposed to the frontend.
test_cases/	Excel workbook containing the project's AI/knowledge-base test cases.

7. AI Integration and Knowledge Base

- The system prompt defines Solari as a solar engineering assistant focused on Pakistan and establishes strict knowledge-base grounding.
- The prompt instructs the model not to invent technical specifications, prices, regulations, or policies that are absent from the supplied knowledge base.
- The prompt distinguishes power (kW) from energy (kWh) and defines the required inputs for sizing conversations.
- The prompt requires the deterministic calculate_solar_recommendation tool for sizing estimates.
- The prompt includes safety guardrails against unsafe high-voltage DIY instructions and requires professional intervention for hazardous electrical faults.
- The prompt prohibits guarantees of a zero electricity bill and requires a technical/engineering disclaimer for sizing, financial, or hardware recommendations.
- The chat endpoint supplies the system prompt and the concatenated Markdown knowledge base as the AI system instruction.

7.1 Knowledge Sources

- hardware_research.md — solar hardware, panel technologies, and specifications.
- operations_support.md — installation, maintenance, warranties, pricing concepts, customer support, and safety guidance.
- systems_research.md — comparison of on-grid, off-grid, and hybrid PV systems, batteries, inverters, grid interconnection, and Pakistan-focused regulatory material.
- Three PDF knowledge-base documents are also exposed by the frontend: Pakistan Solar Operations Research.pdf, Solar_Hardware_Research.pdf, and Solar_PV_Systems_Report.pdf.

- At startup, the backend loads the three Markdown files into one in-memory knowledge-base string and supplies it to Gemini for grounded responses.

7.2 Gemini Function-Calling Flow

When Gemini determines that a user request requires system sizing, the backend checks the returned function call. If `monthly_units` is available, the backend executes `calculate_solar_recommendation()`, packages the result as a function response, and sends that result back to Gemini. Gemini then produces the final natural-language response. This design reduces the risk of the language model performing unsupported arithmetic on its own.

8. Deterministic Solar Recommendation Engine

The sizing engine in `utils.py` uses fixed project constants and validates the input values before performing the calculation. It returns the original inputs, recommended system size, panel count, panel wattage, roof area, battery capacity, and a disclaimer.

Constant	Configured value
Peak sun hours	4.5 hours/day
Performance ratio	0.80
Panel wattage	550 W
Inverter efficiency	0.95
Battery depth of discharge	0.80
Minimum system size	3.0 kW
Estimated panel area	30 sq ft/panel

8.1 Calculation Formulas

Calculation	Implementation
Daily energy consumption	<code>daily_energy_kwh = monthly_units / 30</code>
Preliminary PV size	<code>preliminary_pv_kw = daily_energy_kwh / (4.5 × 0.80)</code>
Recommended system size	<code>recommended_system_kw = max(preliminary_pv_kw, 3.0)</code>
Panel count	<code>ceil((recommended_system_kw × 1000) / 550)</code>
Roof area	<code>panel_count × 30 sq ft</code>
Battery capacity	<code>((daily_energy_kwh / 24) × backup_hours) / (0.95 × 0.80)</code>

Example interpretation: for 600 kWh/month and 2 hours of requested backup, the implemented formulas produce approximately 5.56 kW recommended PV, 11 panels at 550 W each, 330 sq ft estimated panel area, and about 2.19 kWh battery capacity. This is an application estimate, not a site-specific engineering design.

9. API Specification

Method	Endpoint	Input	Purpose / Output
GET	<code>/health</code>	None	Service health and application name
GET	<code>/</code>	None	Basic application status
POST	<code>/api/recommend</code>	<code>{"monthly_units": 200, "backup_hours": 0}</code>	Deterministic sizing result

Method	Endpoint	Input	Purpose / Output
POST	/api/chat	<code>{"message": "...", "system_type": "on-grid"}</code>	AI response; may include tool_result

9.1 Request Validation

- RecommendationRequest requires monthly_units > 0 and backup_hours >= 0.
- ChatRequest requires a non-empty message and defaults system_type to on-grid.
- Invalid sizing inputs are converted into HTTP 400 responses.
- Unexpected Gemini failures are handled at the API boundary and can produce HTTP 500 responses.

10. End-to-End Data Flow

1. The user enters a solar question in the browser.
2. JavaScript sends a JSON POST request to /api/chat.
3. FastAPI validates the request using Pydantic.
4. The backend checks whether GEMINI_API_KEY is configured.
5. The backend combines the system prompt, knowledge base, and preferred system type.
6. Gemini generates an answer. If sizing is required, Gemini can request calculate_solar_recommendation.
7. FastAPI executes the deterministic Python function with the tool arguments.
8. The tool result is returned to Gemini for a grounded final response.
9. FastAPI returns JSON to the browser.
10. The frontend displays the response, rendering Markdown safely.

The browser therefore does not contain the Gemini API key. The key is read by the backend configuration layer, which is the appropriate architectural boundary for the external AI credential.

11. Security, Safety, and Reliability

- API credentials are intended to be stored in backend/.env rather than committed to source control; backend/.gitignore excludes .env.
- The uploaded project currently contains a credential in .env.example. This is a security issue: the credential should be revoked/rotated immediately and the example file should contain a placeholder only.
- The frontend uses textContent for user messages, reducing direct HTML injection risk.
- AI-generated Markdown is sanitized with DOMPurify before insertion into the page.
- The current CORS configuration uses allow_origins=[*]; production deployment should restrict allowed origins and review credential/CORS settings.
- The backend catches Gemini quota/resource-exhaustion conditions and returns a user-facing temporary-rate-limit message.
- High-voltage electrical operations are intentionally constrained by the system prompt and should be handled by qualified professionals.

11.1 Safety Guardrails

The system prompt explicitly prohibits step-by-step DIY instructions for hazardous high-voltage DC work, requires professional intervention for serious electrical faults, warns against unsafe panel cleaning during peak heat, and prevents the chatbot from promising a completely zero electricity bill. These rules are part of the AI behavior layer and should remain synchronized with the knowledge base and applicable regulations.

12. Configuration and Deployment

The backend README specifies Python 3.10+, a Google Gemini API key, and Uvicorn as the application server.

12.1 Standard Setup

1. Install Python 3.10 or newer.
2. Open a terminal in the backend directory.
3. Create a virtual environment: `python -m venv .venv`
4. Activate the environment (Windows: `.venv\Scripts\activate`; macOS/Linux: `source .venv/bin/activate`).
5. Install dependencies: `pip install -r requirements.txt`
6. Create `backend/.env` and set `GEMINI_API_KEY` and, if required, `GEMINI_MODEL`.
7. Start the API: `uvicorn main:app --host 0.0.0.0 --port 8000 --reload`
8. Open the frontend through a local web server rather than relying on `file://` when possible.
9. For deployment, replace the hard-coded 127.0.0.1 API URLs with the deployed backend URL and configure CORS for the real frontend origin.

12.2 Windows Helper Script

`backend/run.bat` automates creation of the virtual environment, activation, dependency installation, and startup of Uvicorn. It is intended for Windows development.

12.3 Production Considerations

- Run without `--reload` in production.
- Serve the frontend through a proper web server or static hosting layer.
- Use HTTPS and a stable backend hostname.
- Configure the frontend API base URL for the deployment environment.
- Restrict CORS to known frontend origins.
- Keep the Gemini credential outside the repository and rotate any credential that has been exposed.

13. Validation and Testing

- The backend Python source was syntax-checked successfully with `Python compileall`.
- The project includes an Excel workbook named `SolarWise_AI_Final_LLM_Knowledge_Base_Testing.xlsx`.
- The workbook contains a Test Cases sheet with 30 test cases plus a Summary - Metrics sheet.
- The test workbook includes categories such as product questions and sizing and records expected/actual responses and status.
- The technical documentation does not treat the workbook's initial pending metrics as evidence of final production readiness; actual execution results should be recorded after a complete test run.

14. Current Limitations and Technical Risks

- The sizing engine is an initial estimation model, not a full engineering design or site-survey tool.
- The calculator currently uses fixed engineering constants rather than location-specific irradiance, roof orientation, shading, temperature, load curves, or manufacturer-specific electrical limits.
- The battery estimate is based on average daily consumption and backup hours; it does not model individual appliance loads, surge power, battery chemistry, or battery aging.
- The AI depends on the configured Gemini API and can fail when credentials are missing, quota is exhausted, or the external service is unavailable.
- The frontend is configured for local development and requires endpoint/environment changes for production hosting.
- The current backend CORS policy is intentionally broad and should be tightened for deployment.

- The knowledge base is loaded into memory at startup; changes to Markdown files require an application restart to be reflected.

15. Recommended Technical Improvements

- Move frontend API base URLs into a configurable environment or deployment configuration instead of hard-coding 127.0.0.1:8000.
- Replace the credential in .env.example with a placeholder and rotate any credential that has been exposed.
- Pin dependency versions for reproducible builds and document the supported Python version.
- Restrict CORS to the deployed frontend origin(s).
- Add automated unit tests for all sizing formulas and boundary cases.
- Add structured logging, request IDs, and health/readiness checks for deployment.
- Add authentication/authorization if the Leads, administrator, or other protected features become functional.
- Add rate limiting and request-size controls around the public AI endpoint.
- Add a proper deployment architecture for the frontend and backend, including HTTPS.
- Keep regulatory knowledge and engineering assumptions versioned with dates so the chatbot can be updated safely.

16. Conclusion

Solari uses a clear separation of concerns: the frontend handles user interaction and presentation, FastAPI provides the application boundary, the knowledge base grounds domain responses, Gemini provides natural-language reasoning and tool selection, and a deterministic Python engine handles the numerical solar recommendation. This combination is suitable for a prototype and academic/project demonstration. Before production use, the main priorities are credential rotation, configurable API endpoints, tighter CORS, dependency pinning, automated tests, authentication/rate limiting where required, HTTPS deployment, and continued versioning of the solar knowledge base and regulatory assumptions.

Appendix A — Project File Map

Path	Description
backend/main.py	FastAPI application and Gemini orchestration
backend/utlils.py	Sizing engine, knowledge loader, tool schema
backend/config.py	Configuration and environment loading
backend/requirements.txt	Python dependencies
backend/prompts/system_prompt.txt	AI system behavior and safety rules
backend/data/*.md	Primary Markdown knowledge sources
backend/knowladge_base/*.pdf	Reference PDFs
backend/test_cases/*.xlsx	AI/knowledge-base test workbook
frontend/index.html	Web UI structure
frontend/css/style.css	Frontend styles
frontend/js/script.js	Frontend behavior and API integration
backend/run.bat	Windows development startup helper

End of Technical Documentation — Solari